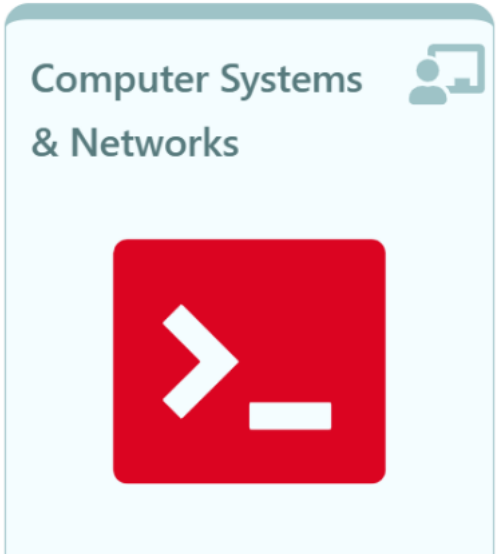
Computer Systems & Networks



SPRINGBOARD
www.springboardcourses.ie

Ireland's European Structural and Investment Funds Programmes 2014-2020
Co-funded by the Irish Government and the European Union

HEA HIGHER EDUCATION AUTHORITY
AN tÚDARÁS um ARD-OIDEACHAS

European Union
Investing in your future
European Social Fund

SETU
Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

Quoting

Single Quote '

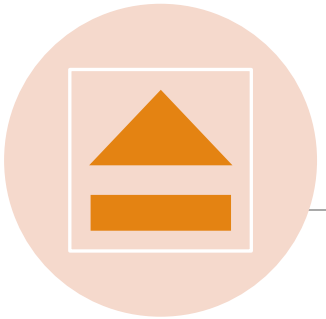
- PREVENT THE SHELL FROM DOING **ANY INTERPR** CHARACTERS:

  EVERYTHING IS TAKEN AS-IS
  E.G. GLOBS, VARIABLES, COMMAND SUBSTITUTIO METACHARACTERS.

```
compsys@compsys-virtualbox:~$ echo The car costs $100
The car costs 00
compsys@compsys-virtualbox:~$ echo 'The car costs $100'
The car costs $100
```

Double Quotes "

Glob characters, also called ***wild cards***, are symbols that have special meaning to the shell (i.e, `*`, `?`).



Double Quotes stop the shell from *interpreting* some metacharacters, i.e.

- glob character * won't expand

Double Quotes

- ✅ preserve spaces

echo "My home is $HOME"

- ✅ allow variable expansion ($var)
- ✅ allow command substitution ($(...))

```
compsys@compsys-virtualbox:~/tmp/tutorial$ echo The glob chars are * ? and []
The glob chars are Dec Oct Sept ? and []
compsys@compsys-virtualbox:~/tmp/tutorial$ echo "The glob chars are * ? and []"
The glob chars are * ? and []
```

Backslash Character \

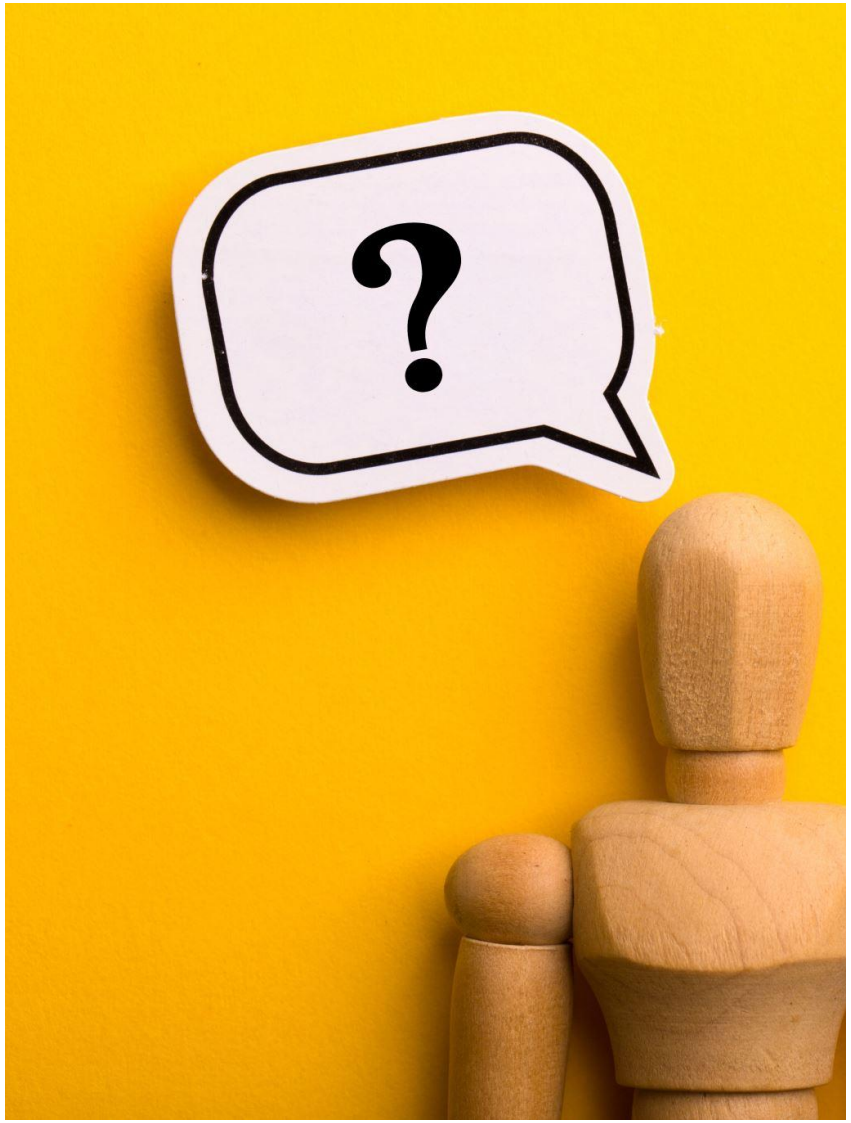
= ESCAPE JUST ONE CHARACTER = SINGLE QUOTE A SINGLE CHARACTER

- IF THE PHRASE BELOW IS PLACED IN QUOTES, `$1` AND `$PATH` ARE NOT VARIABLES:

```
compsys@compsys-virtualbox:~$ echo "The service costs $1 and the path is $PATH"

The service costs  and the path is
/usr/bin/custom:/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
:/usr/games
```

- WHAT IF YOU WANT TO HAVE `$PATH` TREATED AS A VARIABLE AND `$1` NOT?

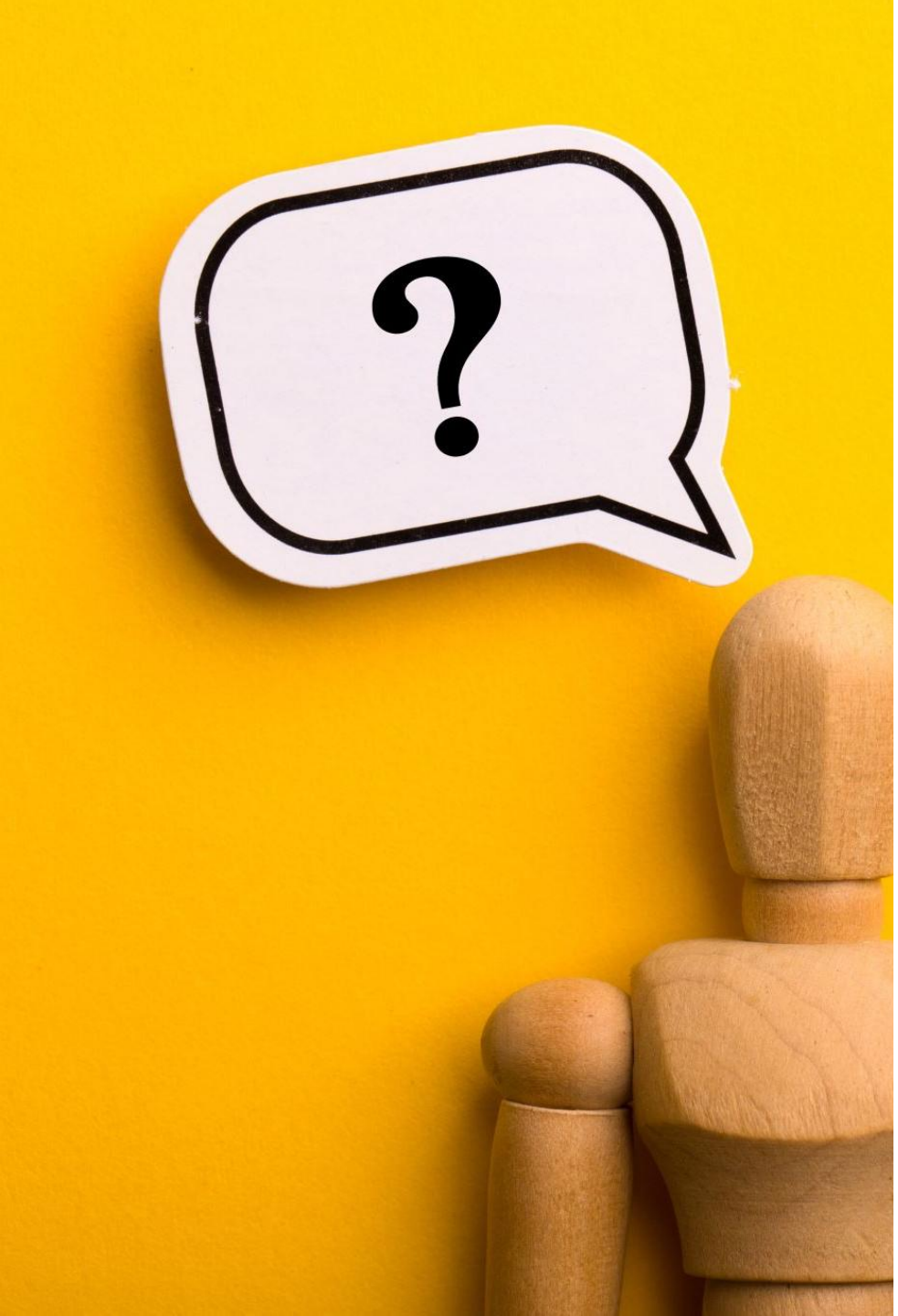
```
compsys@compsys-virtualbox:~$ echo The service costs \$1 and the path is $PATH

The service costs $1 and the path is
/usr/bin/custom:/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bi
n:/usr/games
```



Single or Double Quotes?

SINGLE '

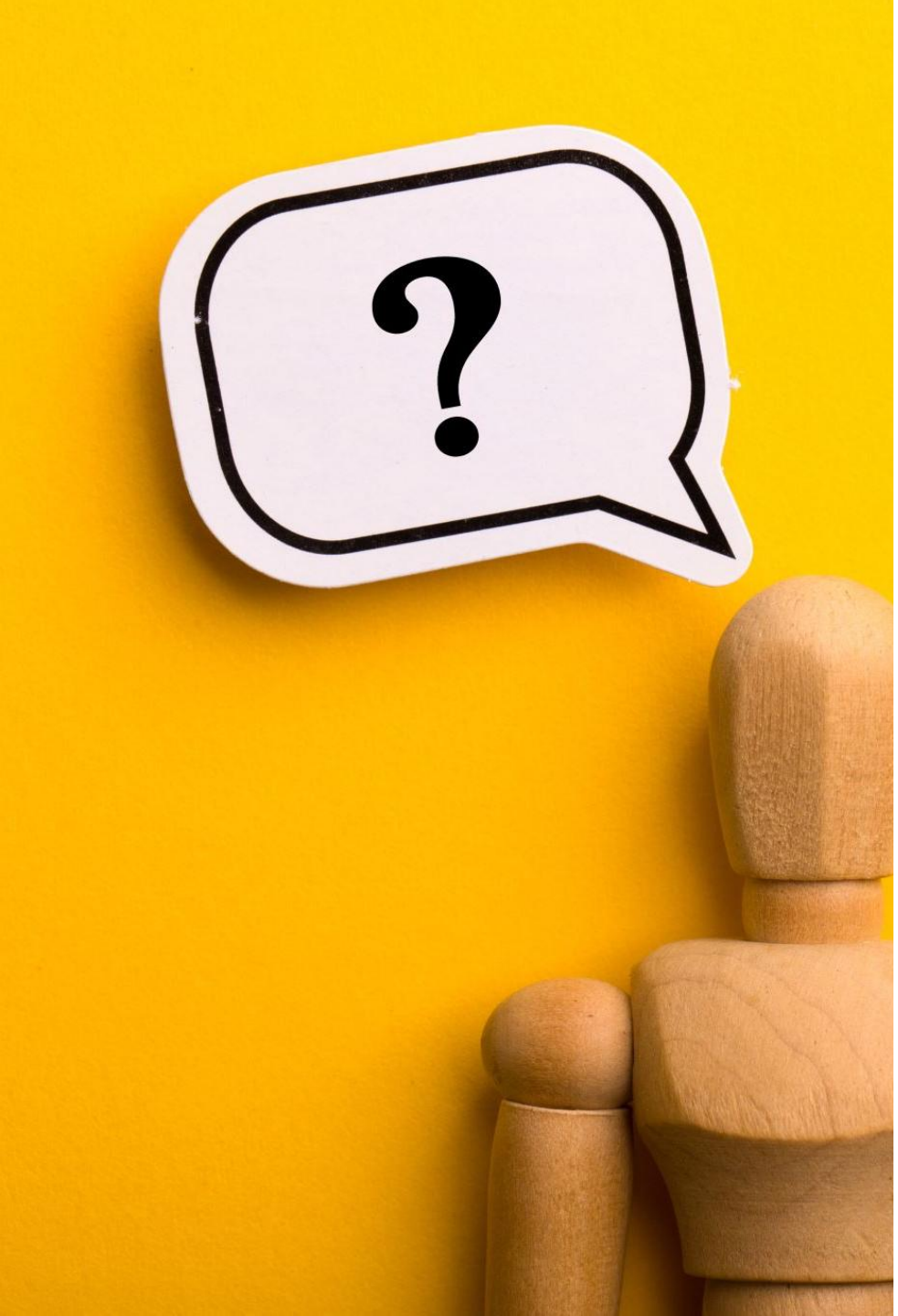
- literally echo content
- Content is printed as it is.



Single or Double Quotes?

DOUBLE "

- will evaluate & output the value of the variable
- *Common Use*: Preserves spaces



🤔 But it works without quotes

📕 **The golden rule**

Quote unless you have a specific reason not to.

[have a look at $man bash]

Command substitution using $(...)

When you want to run the command inside the parentheses, then replace the whole $(...) with that command's output

$echo date

$echo "$(date)"

Almost always wrap command substitution in double quotes

To prevent errors if your output has multiple words

TRY THESE ON YOUR COMMAND LINE NOW

(Legacy) Backquotes

```
compsys@compsys-virtualbox:~$ echo Today is date
Today is date
```

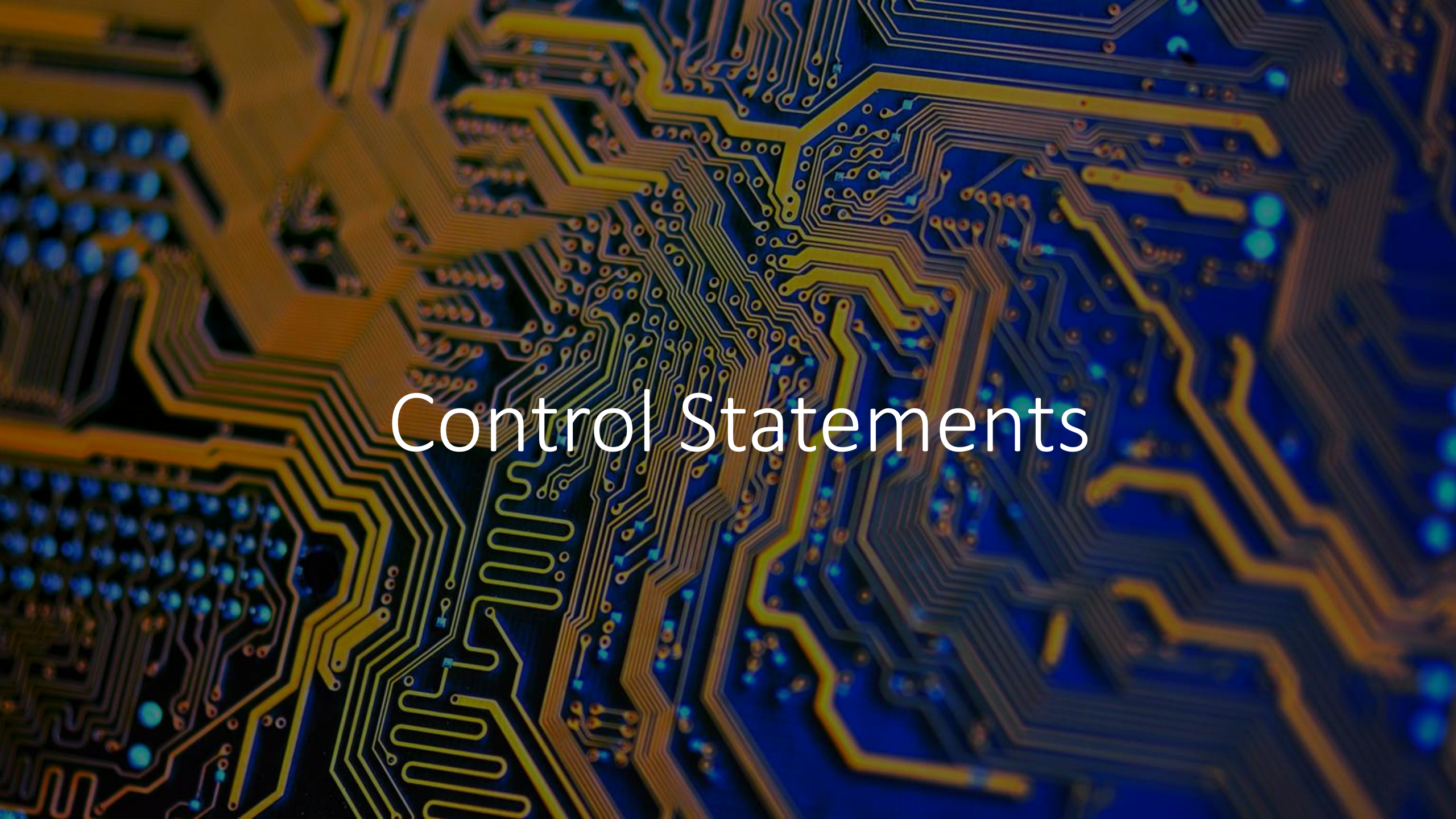
Backticks `...` is outdated method to perform **command substitution**

$echo `$date`

=

1. Run date, then
2. Capture its output
3. Replace the backticks with that output

```
compsys@compsys-virtualbox:~$ echo Today is `date`
Today is Wed 1 Sept 2025 12:40:04 IST
```


Control Statements

Control Statements

Control statements allow you to use multiple commands at once or run additional commands.

Control statements include:

- Semicolon (;)
- Double ampersand (&&)
- Double pipe (||)

The semicolon can be used to run **multiple** commands, one after the other

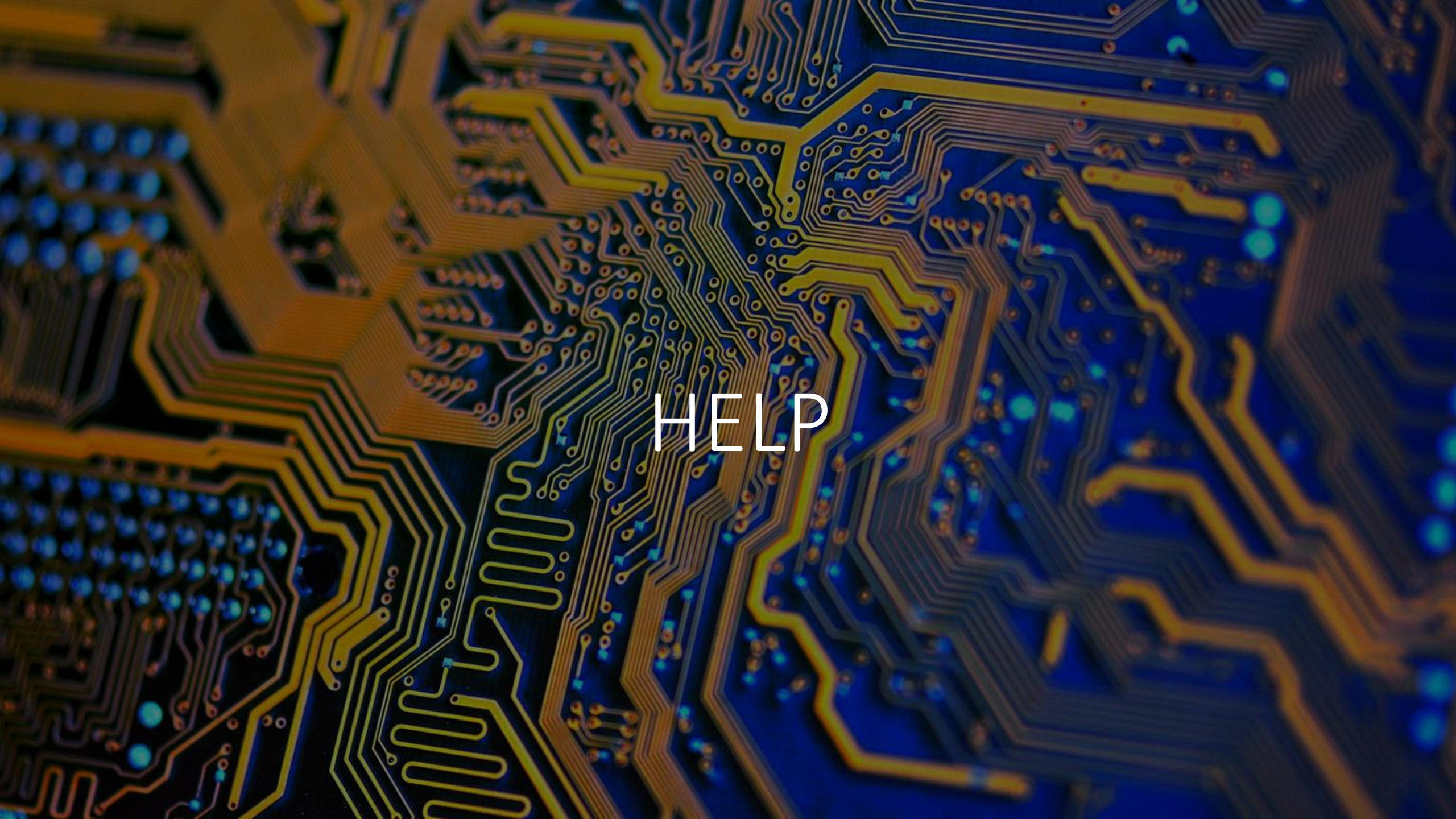
```
compsys@compsys-virtualbox:~$ cal 1 2025; cal 2 2025; cal 3 2025
```

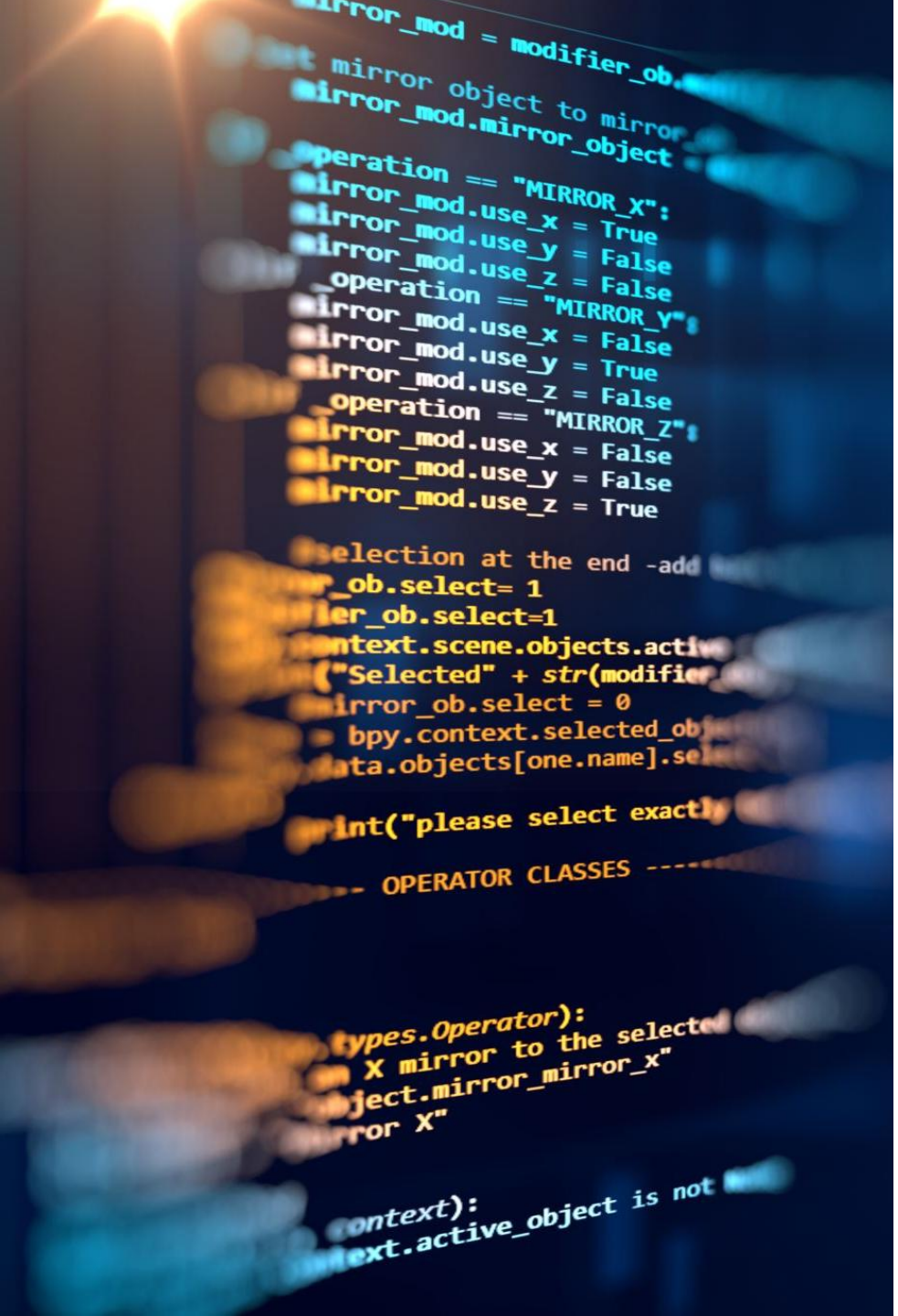
The double ampersand && acts as a logical "**and**" if the first command is **successful**, then the second command (to the right of the &&) will also run:

```
compsys@compsys-virtualbox:~$ ls /etc/perl && echo That listed it perfectly
Net
That listed it perfectly

cess
```

The double pipe || is a logical "or". It works similarly to &&; depending on the result of the first command, the second command will either run or be skipped:

```
compsys@compsys-virtualbox:~$ ls /etc/xml || echo failed
ls: cannot access /etc/xml: No such file or directory
failed
```


HELP



manual = Help Pages

- UNIX IS THE OPERATING SYSTEM THAT LINUX WAS MODELLED AFTER.
- THE DEVELOPERS OF UNIX CREATED HELP DOCUMENTS CALLED *MAN PAGES* (SHORT FOR *MANUAL PAGE*).
- MAN PAGES PROVIDE A BASIC DESCRIPTION OF THE PURPOSE OF THE COMMAND, AS WELL AS DETAILS REGARDING AVAILABLE OPTIONS.

Searching Man Pages

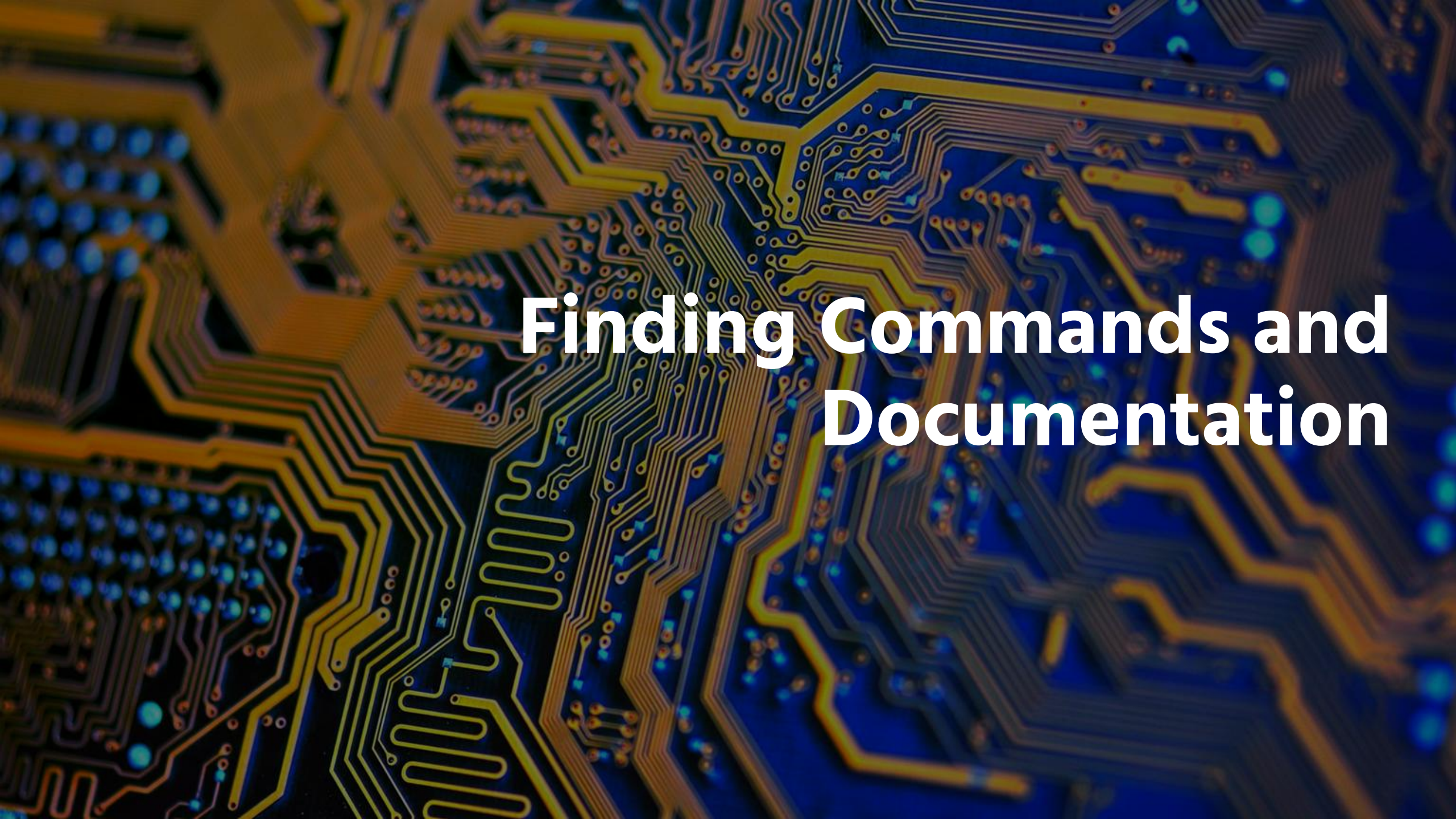
```
/all
```

To search a man page for a term, press the / and type the term followed by the **Enter** key.

If a match is found:

Type **n** to move to the **next** match of the term

Type N to return to a previous match of the term



Finding Commands and Documentation

Finding Commands and Documentation

use the **whereis** command to search for:

- the location of a command

or

- the man pages for a command

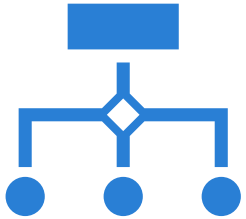
This command searches for commands, source files and man pages in specific locations where these files are typically stored:

```
compsys@compsys-virtualbox:~$ whereis ls
ls: /bin/ls /usr/share/man/man1p/ls.1.gz /usr/share/man/man1/ls.1.gz
```

Man pages are easily distinguished from commands as they are typically compressed with a program called **gzip**, resulting in a filename that ends in **.gz**.

Find Any File or Directory

```
compsys@compsys-virtualbox:~$ locate -c passwd
97
```



To find any file or directory, use the locate command.

The output can be quite large so it may be helpful to use the following options:

-c will list how many files match:

-b only includes listings that have the search term in the *basename* of the filename.

Try place a **** character in front of the search term:

```
compsys@compsys-virtualbox:~$ locate -b "\passwd"
```